



ENSEEIH - INP Toulouse
A/R 2ème Année – Semestre 8 – 2024-2025
Final Project's Description

Course: N8EN18B: Contrôle et Apprentissage

Due date: 18/03/2025 **Weight:** 100 %

General Instructions:

- Submit your project files in the appropriate section on Moodle.
- Any cheating or plagiarism attempt will result in the group's disqualification.
- The grade will be assigned to the group and shared by all members

	Part I: Report			Part II: Demo	Total
	Activity 1: Problem Statement	Activity 2: Agent Modeling	Activity 3: Algorithm Design	Activity 4: Performance Evaluation	
Value	2.0	4.0	4.0	10.0	20.0
Grade					

In this project, you are supposed to introduce a problem of interest and propose a solution based on Reinforcement Learning algorithms. The project has two main parts:

- Part (I) Report: You will describe a few activities related to the theoretical and design aspects of your project. See Section 1 for more details.
- Part (II) Demo: You will show your code in a Python notebook and discuss the obtained results. See Section 2 for more details.

Your submission must contain exactly two files: (1) a PDF file for part I and (2) a Python notebook file for part II.

1 Part I: Report

The report must be written using IEEE conference template available here and structured in sections. Although you are free to organize your text however you prefer, your report must contain the following sections.

1.1 Problem Statement

Provide context and relevant details to the problem’s description. Make sure your problem is clearly defined as a sequential decision-making task. Use diagrams or examples to illustrate the dynamics of the environment. At the end, answer the questions below:

- What is the environment (simulated or real)? Describe it.
- What is the agent’s objective? What defines success?
- Why is RL a suitable approach?
- Is the problem episodic or continuous?
- Are there any known benchmarks or comparison baselines?

1.2 Agent Modeling

Describe how you formulated the agent’s interaction with the environment. Questions to answer:

- What is the state representation? How is it encoded (raw features, embeddings, etc.)?
- What are the action types (discrete, continuous)? How are actions represented?
- How is the reward function defined?
- Is the environment fully observable (MDP) or partially observable (POMDP)? How did that influence your design?
- What is the time horizon? Did you use a discount factor γ , and how did you choose it?

1.3 Algorithm Design

Explain the RL algorithm you chose, how it was implemented, and why you believe it’s suitable for your problem. Try to justify your choices both empirically (based on results) and theoretically (based on the literature).

- Which algorithm did you choose (e.g., DQN, Policy Gradient, SARSA, Q-Learning, Dynamic Programming, etc.)? Why?
- Did you model the policy as deterministic or stochastic?
- Is it an on-policy or off-policy algorithm? What are the implications for your training?
- Did you use exploration strategies (e.g., epsilon-greedy, entropy regularization)? How did you choose the parameters?
- How did you tune hyperparameters (learning rate, batch size, update frequency, etc.)?
- Did you use target networks, experience replay, or reward normalization?
- Did you use a function approximator (e.g., neural network)? What architecture did you choose and why?

2 Part II: Demo

Your demo will be developed on a Python notebook, and it should demonstrate the implementation, training, and evaluation of your (D)RL agent. The notebook must be organized, annotated, and reproducible. Each notebook should contain the following sections:

2.1 Coding Environment Setup

Set up the coding environment and necessary packages.

Must include:

- Installation and import of required libraries (‘gym’, ‘torch’, ‘stable-baselines3’, etc.).
- A brief description or link to the environment used.
- Environment wrapper(s), if any, and the reason for them.

2.2 Agent Design and Implementation

Implement your RL agent (or integrate an existing algorithm with clear explanation of modifications).

Must include:

- Clear code for policy/value networks.
- Training loop or call to a training method.
- Description of architecture and key hyperparameters in Markdown cells.

2.3 Training Process

Train your agent and visualize its learning progress.

Must include:

- Code for training the agent over multiple episodes.
- Plots of reward over time (e.g., moving average).
- Training time or number of steps/episodes.

2.4 Evaluation and Performance

Evaluate the final trained agent and demonstrate its behavior.

Must include:

- Code to run several test episodes without exploration noise.
- Average return or success rate over multiple evaluation runs.
- Visualizations [[If applicable]]: trajectory plots, rendered videos (if possible), performance histograms.

2.5 Submission Requirements

- Submit a single ‘.ipynb’ notebook.
- Include all necessary code to run from start to finish.
- Add clear comments and markdown explanations throughout.
- Plots and tables must be labeled and interpreted.
- If using stochastic policies, seed your code for reproducibility.
- Include a link to your GitHub repo or zip file with additional resources if needed.